

COMP 3069

Computer Graphics



Lecture 7:
Render Pipeline
Autumn 2024



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

Contents

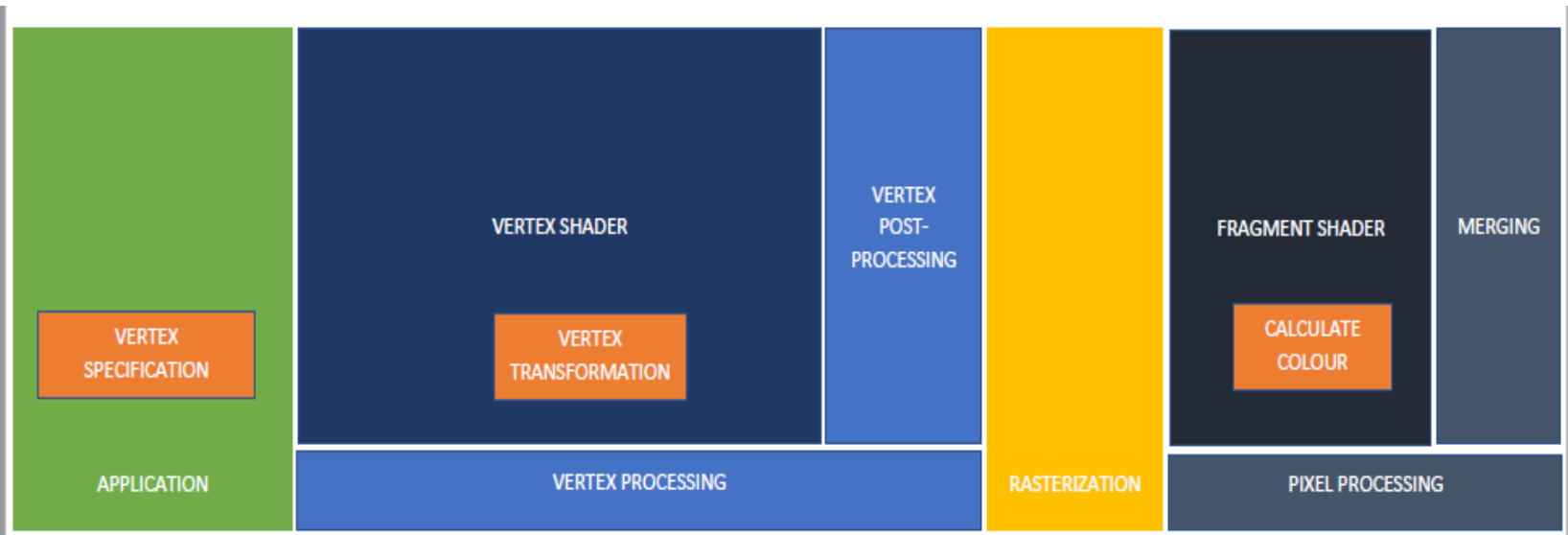
- ◆ Describe the overall Rendering Pipeline
- ◆ Visualise vertex specification, vertex shading, rasterisation and fragment shading
- ◆ Describe clipping
- ◆ Describe merging

Rendering Pipeline

渲染流水线--四大阶段

它们把在 CPU 里准备的顶点/纹理等数据，一步步变成屏幕像素。

- ◆ Application 应用层
- ◆ Vertex processing 顶点处理
- ◆ Rasterisation 光栅化
- ◆ Pixel Processing 像素/片段处理



Application Stage

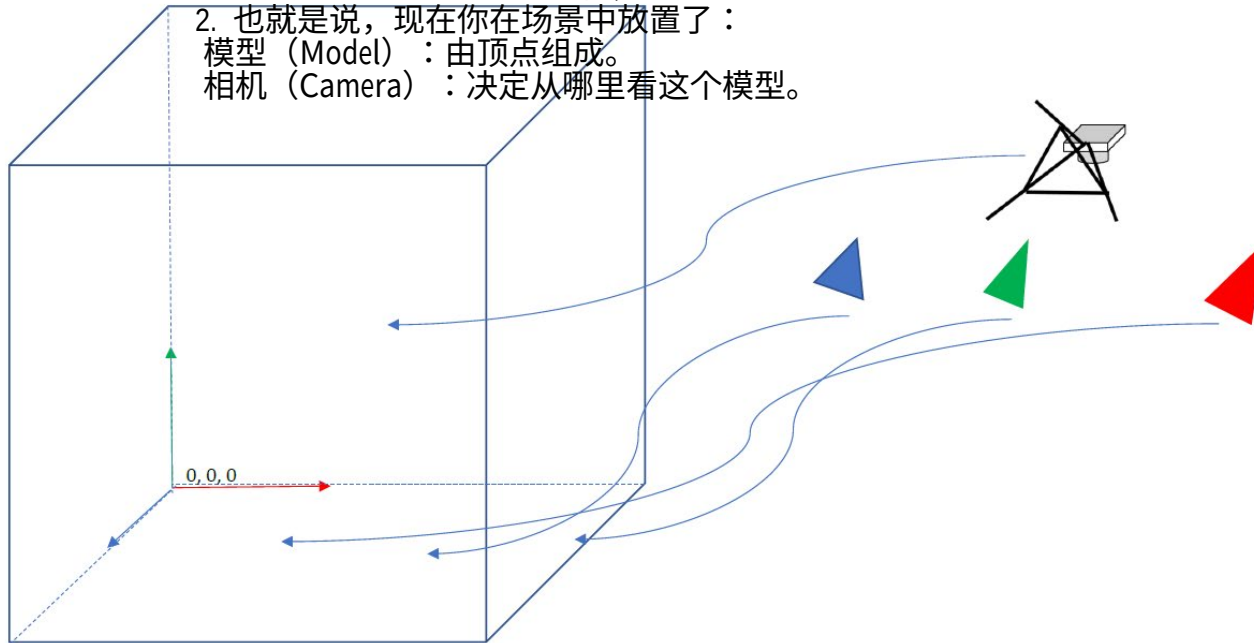
- ◆ Running software
- ◆ Writing code with a `main()` function
- ◆ Using `glBufferData()` to copy the vertices data to the OpenGL data storage
- ◆ Handling user input and update model positions



Vertex Specification 顶点规范

- ◆ The vertex specification happens in the application 应用程序
- ◆ An array of vertices is created 创建顶点数组
- ◆ A camera and vertices have been specified at certain locations

1. 这句话意味着：
你不仅定义了物体的顶点坐标，还定义了相机的位置和方向。
2. 也就是说，现在你在场景中放置了：
模型（Model）：由顶点组成。
相机（Camera）：决定从哪里看这个模型。



Vertex Processing Stage

顶点处理阶段

对每个顶点进行独立的处理操作

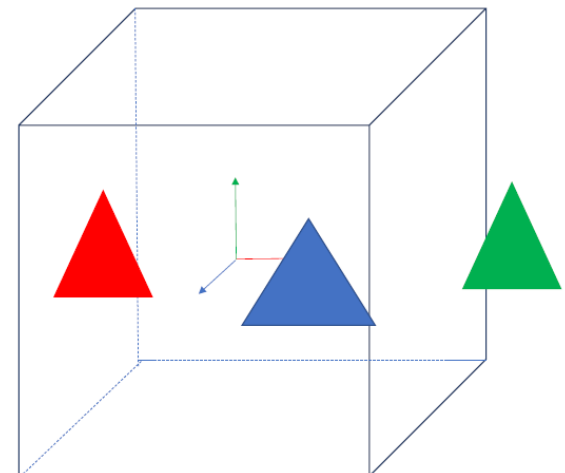
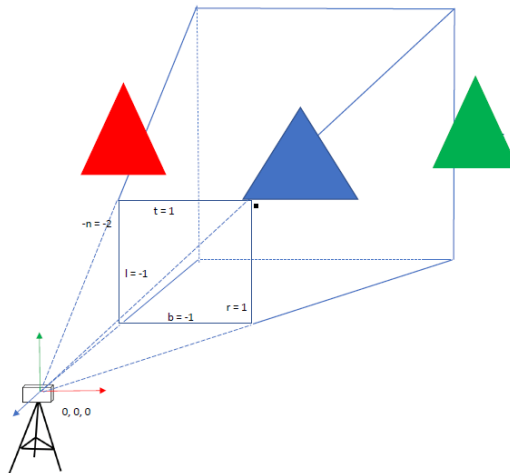
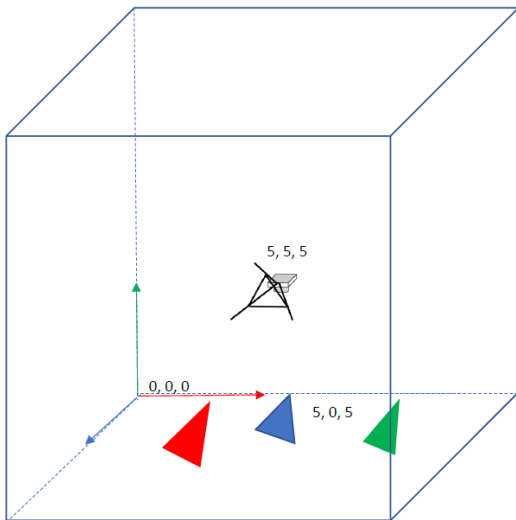
- ◆ Processing operations on individual vertices
顶点变换: model space-->world space-->view space-->clip space: $P_{clip} = P * V * M * P_{model}$
- ◆ Transforming vertex positions from model space to world space and view space, and projecting them to clip coordinates
裁剪不可见的三角形
- ◆ Clipping triangles that are not visible
透视除法: 将顶点位置转换成标准设备坐标系-->clip space-->NDC space
- ◆ Performing perspective division to transform vertex positions into normalised device coordinates
NDC space--> screen space
- ◆ Transforming x and y components to screen coordinates and storing z components in a depth buffer



Vertex Shader 顶点着色器

- ◆ After transformations, the vertices are now in a cube called the view volume, which represents what is actually visible to the camera

当所有坐标变换完成后，顶点被映射到一个称为“视见体（view volume）”的立方体中，这个立方体表示相机实际能看到的空间。



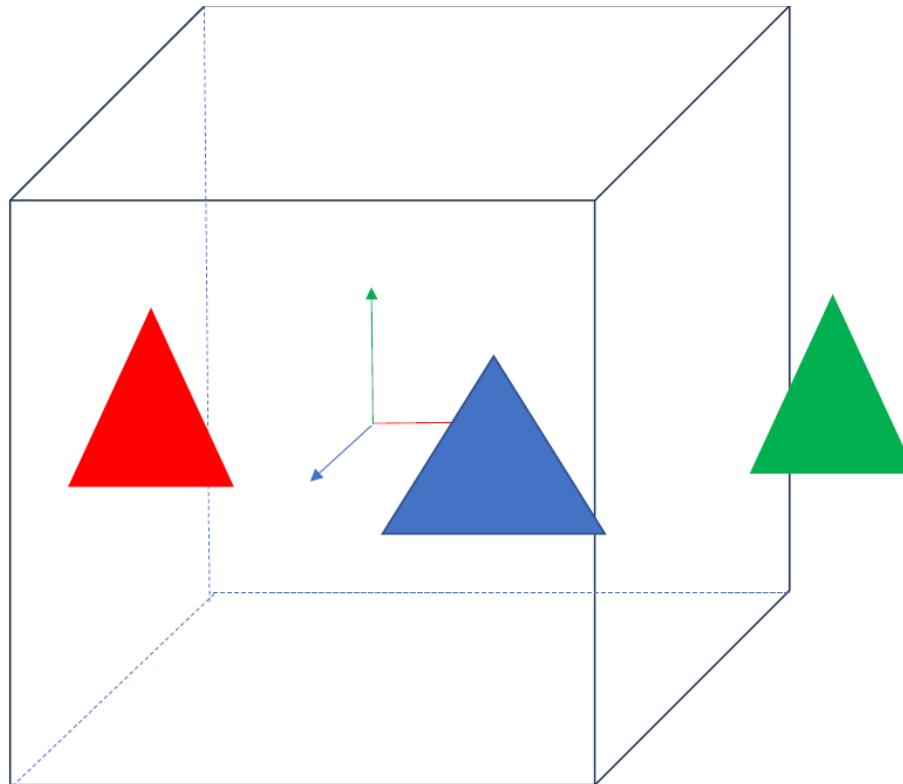
Vertex Post-Processing 顶点后处理

- ◆ Primitive Assembly 图元装配
- ◆ Clipping 裁剪
- ◆ Perspective Division 透视除法
- ◆ Viewport transformation 视口变换

Primitive Assembly

- ◆ After primitive assembly, OpenGL has a list of triangles which are in front of the camera and need to be passed to the next stage for further checking

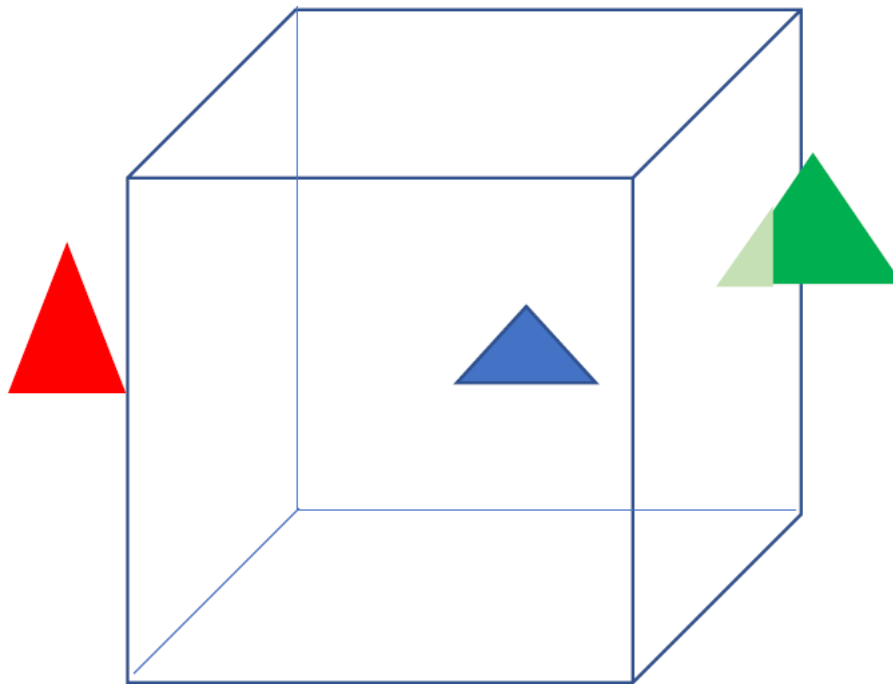
在初步组装之后，OpenGL 会生成一个三角形列表，这些三角形位于摄像机前方，需要传递到下一阶段进行进一步检查。



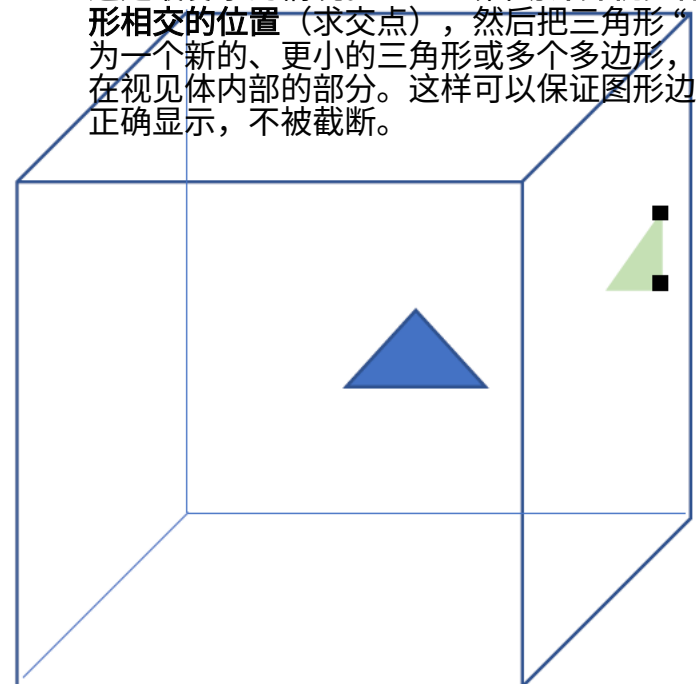
Clipping

◆ There are 3 cases:

- A triangle is completely **inside** the view volume 三角形完全位于视体内部
- A triangle is completely **outside** the view volume 三角形完全位于视体外部
- A triangle **intersects** the view volume 三角形与视体相交

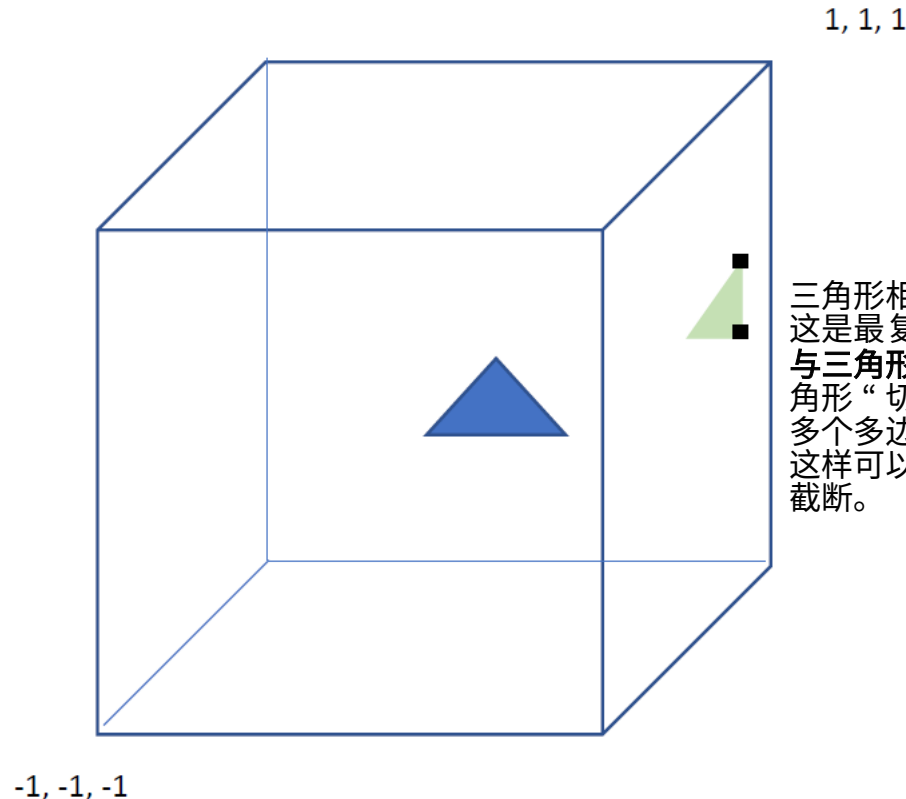


这是最复杂的情况。GPU 会自动计算视见体与三角形相交的位置（求交点），然后把三角形“切割”为一个新的、更小的三角形或多个多边形，只保留在视见体内部的部分。这样可以保证图形边缘依然正确显示，不被截断。



Perspective Division

- ◆ After perspective division transformation, triangles are in Normalised Device Coordinate space 经过透视分割变换后，三角形位于“标准化设备坐标”空间中。



三角形相交：
这是最复杂的情况。GPU 会自动计算视见体与三角形相交的位置（求交点），然后把三角形“切割”为一个新的、更小的三角形或多个多边形，只保留在视见体内部的部分。这样可以保证图形边缘依然正确显示，不被截断。

Viewport Transformation 视口变换

- ◆ After the vertex postprocessing, the triangles are in screen space with an additional z component

经过顶点后处理之后，这些三角形就处于屏幕空间中，并且多了一个 z 分量。



$0, 0$



The University of
Nottingham

Rasterisation Stage 光栅化阶段

计算三角形内部的所有像素。使用重心坐标去计算

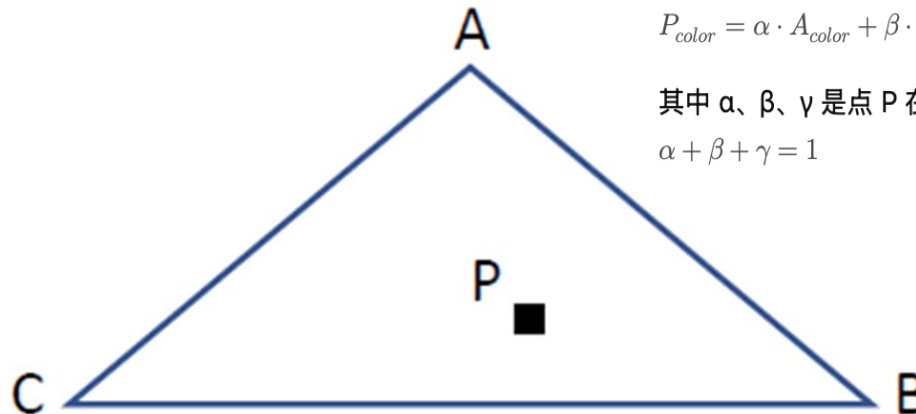
- ◆ All of the pixels inside the triangles are calculated
抗锯齿技术可以为每个像素使用多个采样点。
- ◆ Anti-aliasing techniques can use more than one sample for each pixel
- ◆ Any sample inside a triangle is called a fragment
任何位于三角形内部的采样点都称为一个“片段 (Fragment)”。
- ◆ The fragment attributes (e.g., position, colour, depth) are interpolated from the vertex attributes

例如：

$$P_{color} = \alpha \cdot A_{color} + \beta \cdot B_{color} + \gamma \cdot C_{color}$$

其中 α 、 β 、 γ 是点 P 在三角形内的重心坐标系系数，满足：

$$\alpha + \beta + \gamma = 1$$



Pixel Processing Stage

像素处理阶段

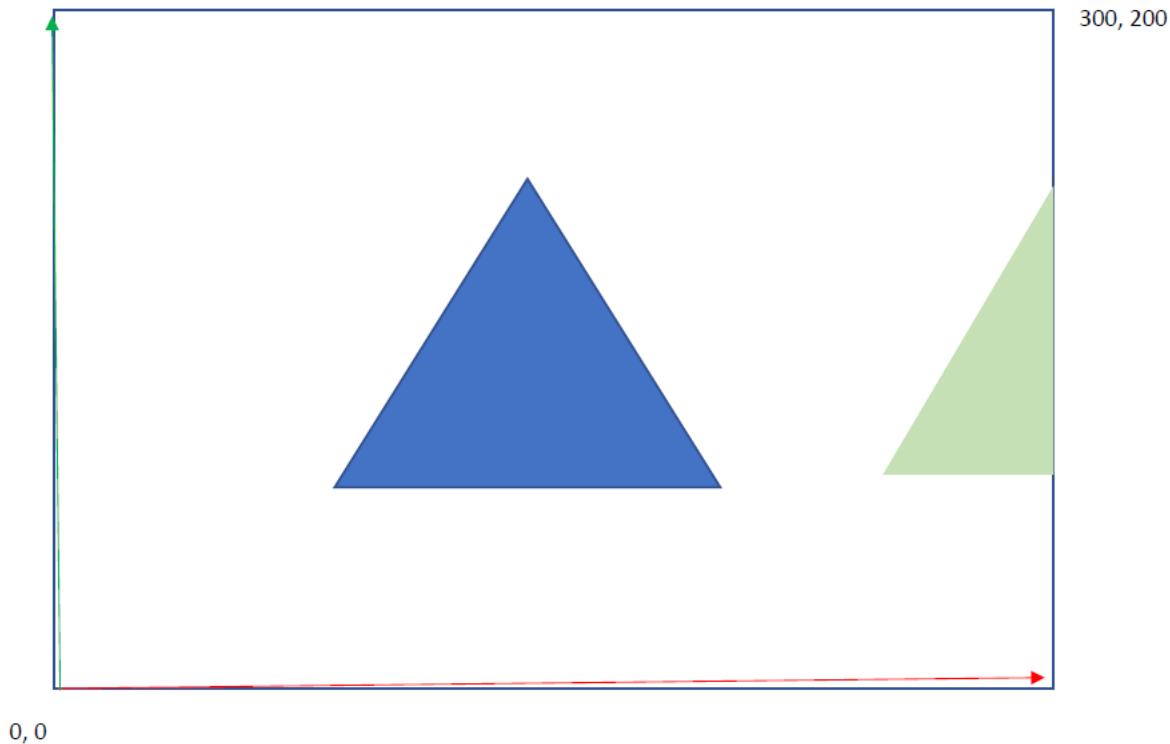
像素处理阶段负责对每一个片段执行操作。

- ◆ The pixel processing stage is responsible for processing operations on individual fragments
纹理贴图操作也会在片段着色器中进行。
- ◆ Texturing can also happen in the fragment shader (see chapter 9)
片段着色器计算出一个颜色值，然后与颜色缓冲区（Color Buffer）进行合并。
- ◆ The **fragment shader** computes a colour which is then merged with the colour buffer
合并（Merging）阶段用于计算哪些片元被遮挡，只保留可见片元的颜色。
- ◆ **Merging** calculates which fragments are hidden behind other fragments and only keeps the colour for the visible fragment



Fragment Shader 片段着色器

- ◆ It is common for the fragment shader to sample texture colours (Chapter 9) or calculate the amount of light reaching a fragment from a light (Chapter 10) 通常，片元着色器要么用于从纹理中采样颜色（参见第 9 章），要么用于计算光照模型中每个片段接收到的光的强度（参见第 10 章）。



Merging 是将片元着色器 (Fragment Shader) 输出的颜色, 与帧缓冲区中已有的颜色 (Color Buffer) 进行合并 (merge), 并根据深度信息 (Z-Buffer) 确定哪些片元可见、哪些被遮挡。

Merging 合并

只有观众 (相机) 能看到的三角形部分应该被显示。

- ◆ Only the parts of triangles visible to the viewer should be displayed

合并阶段将片元着色器生成的颜色, 与颜色缓冲区 (Color Buffer) 中已有的颜色合并。

- ◆ Merging combines the fragment colour produced by the fragment shader with the colour already stored in the colour buffer

可见性 (哪个片元在前、哪个被挡住) 由 Z-buffer (深度缓冲区) 决定。

- ◆ Visibility is resolved using the z-buffer

Z-buffer 的大小与显示区 (Display) 和颜色缓冲区 (Color Buffer) 相同, 并且每个元素在开始时被初始化为 +无穷

- ◆ The z-buffer is the same size as the display and colour buffer, and each element is initialised to

$+\infty$

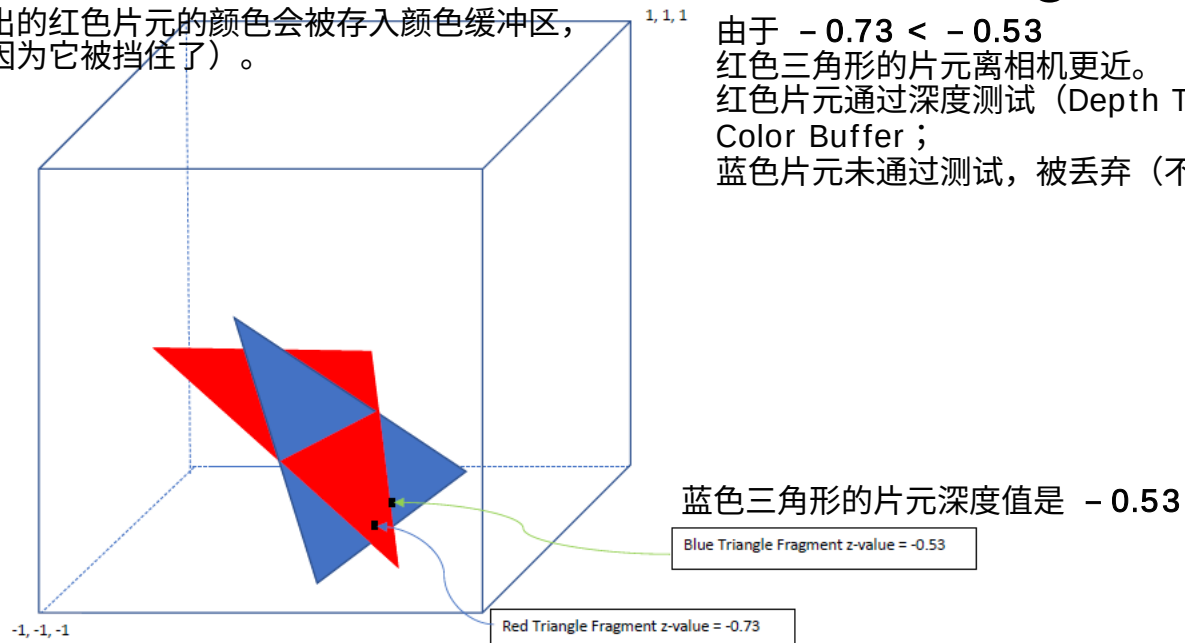


Merging Example

- ◆ A red triangle and a blue triangle are intersecting and the bottom of the blue triangle is behind the red triangle 一个红色三角形和一个蓝色三角形相交，蓝色三角形的底部部分位于红色三角形之后（更远的位置）。
- ◆ When the colour buffer is displayed, the red fragment colour labelled in the figure will be stored in the colour buffer but the blue fragment

当颜色缓冲区被显示时，图中标出的红色片元的颜色会被存入颜色缓冲区，而蓝色片元的颜色不会被存入（因为它被挡住了）。

由于 $-0.73 < -0.53$
红色三角形的片元离相机更近。
红色片元通过深度测试（Depth Test），颜色写入 Color Buffer；
蓝色片元未通过测试，被丢弃（不显示）。

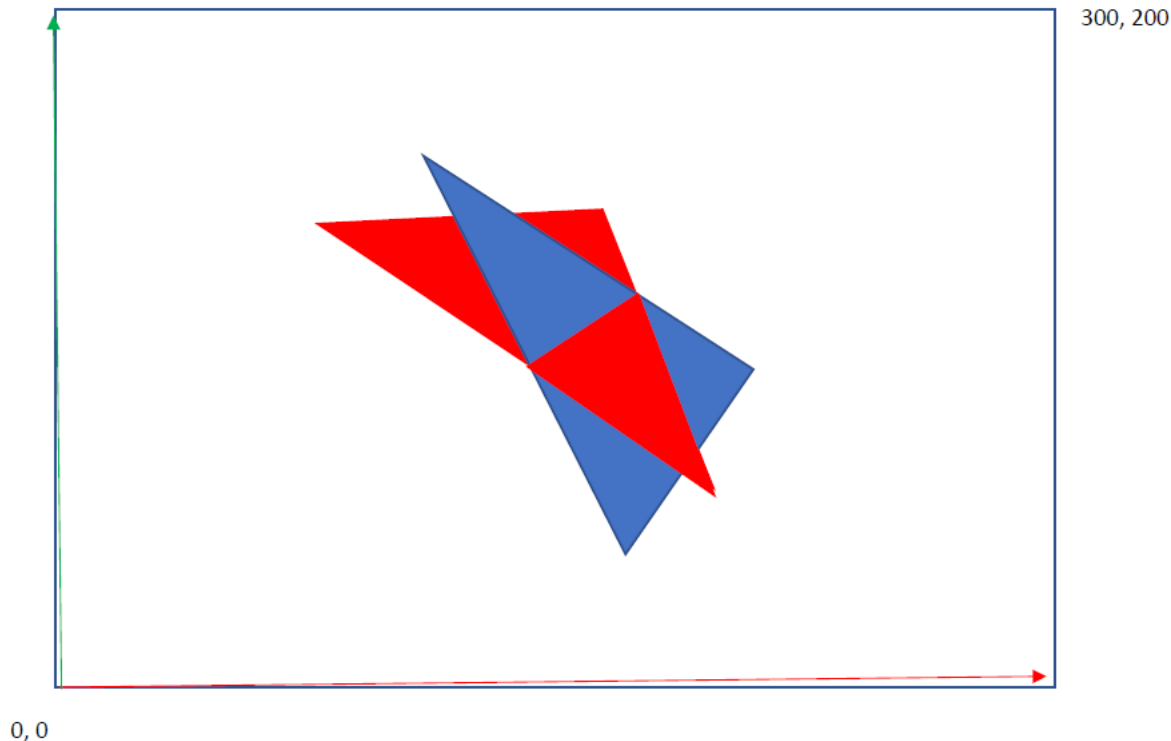


红色三角形的片元深度值（z-value）是 -0.73

Merging Results

在合并阶段之后，只有那些与三角形实际可见部分相对应的片段才会在相应的像素位置上被着色。

- ◆ After the merging stage, only the fragments corresponding to actual visible parts of triangles are coloured at the respective pixel location



References

- ◆ *COMP3069 Lecture notes: Chapter 7 - Graphics Programming, OpenGL & GLSL*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 6*